
Network Access Control (NAC) Sistemi

Teknik Tasarım ve Uygulama Raporu

AAA Mimarisi ve rlm_rest Entegrasyonu

Zeynep Hanife Çelikoğlu

22 Mart 2026

İçindekiler

1	Projenin Amacı	2
2	Sistem Mimarisi ve AAA Entegrasyonu	2
2.1	Mimari Yaklaşım	2
2.2	Authentication (Kimlik Doğrulama)	2
2.3	Authorization (Yetkilendirme)	2
2.4	Accounting (Günlükleme)	3
2.4.1	Accounting-Start	3
2.4.2	Interim-Update	3
2.4.3	Accounting-Stop	3
3	Veritabanı Mimarisi	3
3.1	Kimlik Doğrulama ve Yetkilendirme Tabloları	3
3.2	Accounting ve Performans Katmanı	4
4	Güvenlik Tasarımı ve Veri Standardizasyonu	4
4.1	Veri Standardizasyonu	4
4.1.1	1. Katman: RADIUS Seviyesi (filter_username & preprocess)	4
4.1.2	2. Katman: FastAPI Seviyesi (extract() fonksiyonu)	4
4.2	Güvenlik Değerlendirmesi	4
5	Asenkron Veri Yönetimi ve Bağlantı Havuzlaması	4
6	Dashboard	5
7	Gerçek Dünya Uygulama Senaryoları	5
7.1	Senaryo 1: Denizli Tekstil Endüstrisi Güvenliği	5
7.2	Senaryo 2: BOTAŞ Boru Hattı Güvenliği	5
8	Kişisel Değerlendirme ve Öğrenilenler	5
9	Kaynakça	6

Projenin Amacı

Ağıdaki erişimi ve trafiği güvenli bir şekilde yönetebilmek için geliştirilen mikroservis tabanlı bir NAC sistemidir. Projede amaç, AAA (Authentication, Authorization, Accounting) mimarisini FreeRADIUS ve FastAPI katmanlarına entegre ederek kullanıcılar ve cihazlar için bir ağ yönetim sistemi oluşturmaktır.

Sistem Mimarisi ve AAA Entegrasyonu

Projede FreeRADIUS 3.2, PostgreSQL 18 (Alpine), Redis 8 (Alpine) ve Python 3.12+ tabanlı FastAPI kullanıldı.

Mimari Yaklaşım

FreeRADIUS sadece sunucu olarak değil, RADIUS paketlerini JSON formatına dönüştüren bir *gateway* olarak kullanıldı. Böylece ağ donanımları ile backend (FastAPI, Redis) arasında uyumluluk sağlayan bir köprü oluşturuldu.

Authentication (Kimlik Doğrulama)

Bir kullanıcı sisteme girdiğinde kullanıcı adı ve şifre bilgilerini (IoT cihazı ise MAC adresini) Switch'e yollar. Switch kendi IP adresini ekleyip **Access-Request** paketini RADIUS sunucusuna gönderir. FreeRADIUS, Switch IP'sini **nas** tablosunda arar, eşleşme olmazsa paketi hiç açmadan *discard* eder. Eğer IP kayıtlıysa Shared Secret ile şifreyi çözmeye çalışır, ardından Pre-Validation aşamasında **rlm_rest** modülüyle FastAPI'ye ulaşır. FastAPI, Redis'ten hatalı deneme limitini kontrol eder. Limit aşılmışsa *Reject* döner. Eğer Redis'te bir engel yoksa tekrar RADIUS sunucusuna kimlik doğrulama için geri döner ve sunucu kendi **rlm_sql** modülü üzerinden PostgreSQL'e ulaşır, **radcheck** tablosunda kullanıcı adı ve şifre bilgilerini kontrol eder.

Authorization (Yetkilendirme)

Authentication kısmı başarıyla tamamlandıktan sonra, kullanıcının ağ üzerindeki yetkisini belirlemek için *post-auth* aşamasından devam edilir. FreeRADIUS, **rlm_rest** modülü üzerinden FastAPI'ye doğrulanmış kullanıcının hangi gruba (IoT, misafir, personel) ait olduğunu öğrenmek için ikinci bir sorgu gönderir. FastAPI önce veritabanındaki **radusergroup** tablosundan kullanıcının hangi gruba dahil olduğunu bulur, ardından **radgroupreply** tablosundan o grubun sahip olduğu RADIUS bilgilerini alır ve FreeRADIUS'un anlayacağı bir JSON paketi hazırlar. Bu paketin içine yalnızca VLAN ID'sini değil, Switch'in anlayacağı attribute'ları da ekler. (**Tunnel-Type** işlemin türünü, **Tunnel-Medium-Type** ortamın türünü, **Tunnel-Private-Group-Id** hedef VLAN ID'sini belirtir). FreeRADIUS'un **rlm_rest** modülü, API'den gelen bu JSON verisini alır ve oluşturulmakta olan **Access-Accept** paketinin içine koyar. Ardından bu paketi Shared Secret ile şifreler ve Switch'e yollar. Switch paketi alır, şifresini çözer ve içindeki **Tunnel-Private-Group-Id** değerine göre o porta bağlı olan cihazları belirtilen vlana dahil ederek portu günceller. Böylece cihaz, yalnızca yetkili olduğu ağa erişebilir.

Accounting (Günlükleme)

Accounting-Start

Kullanıcı yetkili olduğu ağ kaynaklarını kullanmaya başladığı an, Switch (NAS) RADIUS sunucusuna bir **Accounting-Request (Start)** paketi gönderir. Paket içinde kullanıcının IP adresi, bağlandığı port, MAC adresi ve benzersiz bir **Acct-Session-Id** bulunur. FreeRADIUS bu paketi alır ve FastAPI'ye (/accounting endpoint'ine) iletir. FastAPI, **INSERT INTO radacct** sorgusunu çalıştırarak oturumun başladığı zamanı (**acctstarttime**) ve cihaz bilgilerini kalıcı olarak veritabanına kaydeder. Redis'de ise **hset** komutuyla **session:{session_id}** anahtarı oluşturulur ve bu ID, **sadd** ile **active_sessions**'a eklenir. Böylece Dashboard'da kullanıcı "Aktif" olarak görünür.

Interim-Update

Oturum devam ederken Switch, 10 dakikada bir güncel trafik bilgisini içeren paketler gönderir. Paket içinde **Acct-Input-Octets** (Gelen Veri) ve **Acct-Output-Octets** (Giden Veri) değerleri bulunur. FastAPI, **UPDATE radacct** sorgusu ile kullanıcının o ana kadarki toplam veri tüketimini ve son görülme zamanını (**acctupdatetime**) veritabanında günceller. Redisdeki canlı oturum bilgisini de günceller.

Accounting-Stop

Kullanıcı kabloyu çektiğinde veya oturum süresi (**Session-Timeout**) dolduğunda Switch bir **Accounting-Request (Stop)** paketi gönderir. Pakette oturumun toplam süresi (**Acct-Session-Time**) ve son veri tüketim miktarları yer alır. FastAPI, PostgreSQL'deki ilgili satırı **UPDATE radacct** sorgusu ile güncelleyerek oturumun bitiş zamanını (**acctstoptime**) yazar ve oturum durumunu "Stop" olarak işaretler. **delete** ve **srem** komutları çalıştırarak aktif oturum kaydı Redis'ten silinir. Böylece Dashboard'da kullanıcı "Çevrimdışı" olarak görünür.

Veritabanı Mimarisi

Kimlik Doğrulama ve Yetkilendirme Tabloları

radcheck: Kullanıcı bilgilerini tutar. Güvenlik için şifreleri **Bcrypt-Password** algoritması ile hashlenmiş şekilde saklar. **idx_radcheck_username** indeksi ile yüksek kullanıcı sayılarında hızlı sorgulama yapılabilir.

radusergroup: Kullanıcıyı (veya MAC adresini) bir yetki grubuna (**groupname**) bağlar. **priority** alanı sayesinde bir kullanıcının birden fazla grubu olması durumunda hangi yetkinin öncelikli olacağı belirlenebilir.

radgroupreply: Dinamik VLAN ataması bu tablo üzerinden yürütülür. **Tunnel-Type** (VLAN), **Tunnel-Medium-Type** (802) ve **Tunnel-Private-Group-Id** (VLAN ID) özellikleri burada tanımlanarak Switch'e gönderilecek **Access-Accept** paketine eklenir.

mac_whitelist: MAB için özel olarak eklenen bu tablo, IoT cihazlarının MAC adreslerini ve açıklamalarını tutar. Tanımlı bir MAC adresi, **radusergroup** tablosu üzerinden ilgili gruba bağlanarak IoT cihazların şifresiz ama kontrollü erişimi sağlanır.

nas: Sisteme erişim isteği gönderebilecek yetkili ağ cihazlarının (Switch/AP) IP adreslerini ve Shared Secret tanımlarını yönetir.

Accounting ve Performans Katmanı

radacct: Oturum sürelerini (`acctstarttime` ve `acctstoptime`), IP adreslerini, trafik verilerini (`acctinputoctets` ve `acctoutputoctets`) ve cihazların MAC adresini (`callingstationid`) tutan tablodur.

Performans İndeksleri: `radacct` tablosunda kullanıcı adı üzerine eklenen `idx_radacct_username` indeksi, geçmişe dönük kullanıcı raporlarının dashboard üzerinden listelenmesini sağlar.

Güvenlik Tasarımı ve Veri Standardizasyonu

Veri Standardizasyonu

1. Katman: RADIUS Seviyesi (`filter_username` & `preprocess`)

Authentication sürecine başlamadan önce `filter_username` ve `preprocess` ile format düzenlemesi yapılır. `filter_username`, ileride PostgreSQL veya FastAPI tarafında sorun oluşmaması için kullanıcı adındaki boşlukları temizler ve rakam ile alt çizgi gibi karakterlerin geçmesine izin verir. `preprocess` modülü ise cihazlardan gelen farklı formatlardaki kullanıcı adını FreeRADIUS'un anlayacağı standart bir formata dönüştürür.

2. Katman: FastAPI Seviyesi (`extract()` fonksiyonu)

kullanıcı doğrulama başarıyla sonra, Accounting verileri Switch'ten FastAPI'ye ulaştırıldığında yapılır. RADIUS paketleri `rlm_rest` ile API'ya nested bir JSON yapısında iletilir. `extract()` yardımcı fonksiyonu bu karmaşık yapıyı (dictionary ve liste) string'e dönüştürür. `Acct-Session-Id`, `User-Name` ve `NAS-IP-Address` gibi verileri garanti altına alır.

Güvenlik Değerlendirmesi

Veri Gizliliği: Kullanıcı şifreleri veritabanında düz metin şeklinde tutulmamalı. Bcrypt algoritmasıyla salt'lanarak hashlenmiş şekilde saklanıyor. (salt, aynı şifreye sahip kişilerin bile veritabanında farklı hash değerlerine sahip olmasını sağlar). Bu sayede veritabanına yetkisiz erişim olsa dahi şifreler ele geçirilemez.

Kaba Kuvvet Koruması: FastAPI ve Redis sayesinde hatalı giriş denemeleri limit aşıldığında Redis üzerinden bir bloklama anahtarı oluşturularak kullanıcı girişi engellenir.

Erişim Kontrolü (MAB): IoT cihazları için geliştirilen MAC Authentication Bypass ile sadece whitelist de tanımlı MAC adreslerine izin verilir, yabancı cihazlar doğrudan REJECT edilir.

Asenkron Veri Yönetimi ve Bağlantı Havuzlaması

Sisteme fazla yük bindiğinde de stabil çalışabilmesi için `asyncpg` kütüphanesi ile her istekte yeni bir veritabanı bağlantısı kurmak yerine `create_pool` ile önceden açılmış

bekleyen bağlantılar kullanılarak gecikme azaltılır.

Dashboard

Dashboarddaki aktif olanların listesi, Redisdeki `active_sessions` ve `session:{id}` kullanılarak oluşturuluyor. PostgreSQL disk üzerinde çalıştığı için monitoring için sorgu atmak işlemciyi yorar, Redis ise ram üzerinde çalıştığından sistemi yormadan dashboard güncellenir. Ayrıca dashboard asenkron çalışır ve HTML'deki `setInterval(loadAll, 10000)` fonksiyonu ile her 10 saniyede bir API'ye sorgu atılarak veriler güncellenir. Dashboard'da aktif olanlar, kayıtlı kullanıcılar, aktif vlanlar ve gruplar yer almakta.

Gerçek Dünya Uygulama Senaryoları

Senaryo 1: Denizli Tekstil Endüstrisi Güvenliği

Denizlideki tekstil fabrikalarındaki akıllı dokuma tezgahları, boyama makineleri ve kalite kontrol cihazları genellikle eski nesil ağ kartlarına sahiptir. Bu cihazların güvenliği sağlanmadığında üretim hattı saldırıya açık hale gelebilir. Tekstil makineleri kullanıcı adı ve şifre giremeyeceğinden dolayı MAB kullanarak Switch, cihazın MAC adresini FreeRADIUS'a iletir. FastAPI bu adresi `mac_whitelist` tablosundan doğrular. Cihaz listede yoksa 401 dönerek fabrika ağına erişim engellenir. Doğrulanmış makineler, `radusergroup` tablosundaki eşleşmelerine göre otomatik ilgili vlana atanır. Bu sayede üretim makineleri fabrikanın diğer ağlarından ayrılır. Makinelerin bağlantı durumları Redisdeki `active_sessions`'da tutulur. Fabrika BT sorumlusu, Dashboard üzerinden hangi makinenin hangi Switch portunda aktif olduğunu ve ne kadar veri tükettiğini izleyebilir. Olası bir saldırıda, üretimdeki bir cihaz ele geçirilse bile saldırganın fabrikanın verilerinin bulunduğu diğer vlanlara erişmesi engellenmiş olur.

Senaryo 2: BOTAŞ Boru Hattı Güvenliği

BOTAŞ vana sensörlerine yetkisiz bir cihazın Switch portundan bağlanması güvenlik açığı oluşturabilir. Switch portuna bir cihaz takıldığında MAC adresi `mac_whitelist` tablosunda aranır. Adres listede yoksa FastAPI 401 döner ve FreeRADIUS Switch'e Reject paketi iletir ve port erişime kapatılır. Doğrulanmış cihaz, `radusergroup` tablosundaki tanımına göre ilgili vlana (ör:SCADA) atanır. Bu sayede sensör verileri kurumun genel internet trafiğinden ve idari ağından izole edilerek taşınır. Sensörlerin veri tüketimi, 10 dakika arayla gelen `Interim-Update` paketlerindeki `Acct-Input-Octets` değerleri üzerinden takip edilir. FastAPI bu veriyi Redis'e anlık olarak işler. Dashboard üzerinden de sensörün trafiği izlenebilir. Mevcut sistemin sunduğu canlı veri akışı, ileride bir eşik değeri eklenerek belirlenen limiti aşan cihaz portlarının API üzerinden otomatik olarak kapatılması özelliği eklenebilir.

Kişisel Değerlendirme ve Öğrenilenler

Projeyle başlarken RADIUS protokolü hakkında çok fazla bilgim olmamasına rağmen Docker ve veritabanını bağladığım kapsamlı bir uygulama süreci oldu benim için. Geliştirme aşamasında karşılaştığım sorunlar ve çözümlerim şunlar:

Konteynerler Arası Haberleşme ve DNS: FastAPI'nin Redis ve PostgreSQL konteynerlarına erişimi sorun oldu. localhost üzerinden çalışan bağlantıların, Docker Network'te servis isimleri (`rac_redis`, `rac_db`) üzerinden güncellenmesi gerekiyormuş. Bu süreçte `lifespan` metodundaki bağlantı hatalarını bulmak için `ping` ve `try-except` kullanarak sistemin dayanıklılığını artırmaya çalıştım.

Veri Tiplerinin Standardizasyonu: FreeRADIUS'un `rlm_rest` modülü üzerinden gönderdiği JSON verilerinin iç içe geçmiş ve bazen liste bazen de string formatında gelmesi veritabanı kayıtlarında hatalara yol açtı. Yardımcı fonksiyon(`extract()`) yazarak çözdüm bu sorunu.

Veritabanı Şeması ve Asenkron Uyumu: `radacct` tablosundaki `acctupdatetime` kolonunun eksik olması, asenkron UPDATE sorgularını reject dönüyordu. Veritabanı şemasını ALTER TABLE ile güncelleyip FastAPI'daki `asynccpg` sorgularını yeni yapıya göre düzenledim.

MAB Mantığının Kurgulanması: IoT cihazları kullanıcı adı ve şifre giremediği için MAC adresini kimlik bilgisi olarak kullanan bir bypass mekanizması kurdum. Normal RADIUS akışını bozmadan `is_mac_address()` fonksiyonu ile gelen verinin kullanıcı şifresi mi yoksa MAC adresi mi olduğunu ayırt edip, eğer MAC adresi ise sistemi bypass moduna sokarak cihazı `mac_whitelist` tablosunda sorgulayacak ve sonuca göre accept ya da reject dönecek.

Sonuç olarak, bu projede sadece AAA sunucusu kurmayı değil, farklı teknolojileri nasıl uyumlu bir şekilde ayağa kaldırabileceğimi öğrenmiş oldum.

Kaynakça

- <https://freeradius.org/documentation/>
- <https://fastapi.tiangolo.com/>
- <https://docs.docker.com/>
- <https://www.postgresql.org/docs/>
- <https://github.com/MagicStack/asynccpg>